

ВВЕДЕНИЕ В СЕТЕВУЮ АРХИТЕКТУРУ ИГР

Unity Netcode for GameObjects (NGO)

Евгений Малько • МИРЭА, 2026

Эволюция мультиплеера

1993



DOOM
IPX, LAN

1996



Quake
Client-Server,
UDP

2004



WoW
MMO,
региональные
сервера

2017



Fortnite
Cloud, 100
игроков

2026



ВАШ ПРОЕКТ
Unity NGO

Проблемы мультиплеера

«Четыре всадника апокалипсиса мультиплеера»



LATENCY

20–300ms
задержка пакетов

Москва → 20ms
Владивосток → 150ms



JITTER

Нестабильность
задержки

50ms → 150ms → 45ms
Игрок телепортируется



PACKET LOSS

1–5% пакетов
теряется

UDP не гарантирует
доставку



CHEATING

Клиент врёт
серверу

Без валидации –
игра сломана

TCP vs UDP

CS:GO использует **UDP** — лучше пропустить 1 пакет, чем ждать повторной отправки

TCP

Надёжность:  Гарантирует доставку

Порядок:  Сохраняет

Скорость:  Медленнее

Overhead:  Больше данных

HTTP, загрузка файлов
Заказное письмо с уведомлением

UDP

Надёжность:  Может потерять

Порядок:  Может перемешать

Скорость:  Быстрее

Overhead:  Меньше данных

Игры, стримы, звонки
Обычная открытка

Что такое «пакет»?

NETWORK PACKET (UDP)

// HEADER

Source IP: 192.168.1.10
Destination IP: 192.168.1.20
Port: 7777

// PAYLOAD

Player ID: 42
Position: (5.2, 0.0, 3.7)
Rotation: (0, 90, 0)
Timestamp: 1738171623

// SIZE: ~128 bytes

Расчёт трафика

10 игроков x 60 пак/с x 128 В

= 76.8 KB/sec

100 игроков x 60 пак/с x 128 В

= 768 KB/sec ≈ 6 Mbps!

Каждое движение игрока = пакет
В шутере: 60 пакетов в секунду!

Tickrate и Update Rate

Tickrate — как часто сервер обрабатывает логику

Update Rate — как часто клиент получает данные

Framerate — как часто рисуется картинка

⚠ Client обновляется чаще, чем Server тикает →
Интерполяция!

Примеры из игр

CS:GO офиц. 64 tick

CS:GO pro 128 tick

Fortnite 30 tick

Rocket League 60 tick

Низкий tickrate = «пуля не попала, хотя прицел был на голове»

Высокий tickrate = больше нагрузка, дороже хостинг

Интерполяция и Экстраполяция

ИНТЕРПОЛЯЦИЯ

Сглаживание прошлого

Получили позицию **A (0,0)**

↓ плавно двигаемся ↓

Получили позицию **B (10,0)**

Клиент рисует:

(2,0) (4,0) (6,0) (8,0)

✓ Плавное движение других игроков

ЭКСТРАПОЛЯЦИЯ

Предсказание будущего

Последняя позиция + скорость

pos: (5,0) vel: 2 м/с вправо

↓

Через 0.1с предполагаем: **(5.2, 0)**

Когда придёт реальная позиция — корректируем

⚠ Ошибка = игрок «телепортируется»

Три модели мультиплеера

P2P

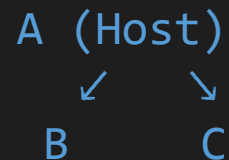
Peer-to-Peer



Все соединены
со всеми

Listen Server

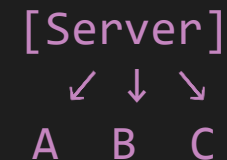
Host-Client



Один игрок =
сервер + клиент

Dedicated

Server



Выделенный
сервер

Выбор топологии — компромисс между **стоимостью**, **защитой** и **сложностью**

Peer-to-Peer (P2P)

// Плюсы

- ✓ Не нужен сервер = экономия
- ✓ Низкая задержка между близкими игроками
- ✓ Просто для 2-4 игроков

// Минусы

- ✗ Не масштабируется (4+ = проблемы)
- ✗ Легко читерить
- ✗ 1 лагает → все страдают
- ✗ NAT traversal

Проблема масштабирования

2 игрока: 1 соедин.

4 игрока: 6 соедин.

10 игроков: 45 соедин.!

Формула: $n(n-1) / 2$

Примеры игр

- Файтинги (Mortal Kombat, Street Fighter)
- Кооп-игры (It Takes Two)

Listen Server (Host-Client)

// Плюсы

- ✓ Звездообразная топология — проще P2P
- ✓ Хост контролирует логику
- ✓ Не нужен выделенный сервер

// Минусы

- ⚠ Хост имеет 0ms преимущество
- ✗ Хост вышел = игра рухнула
- ✗ Зависит от ПК хоста

Unity **NGO** по умолчанию:
`NetworkManager.Singleton`
`.StartHost();`

Когда использовать

- Кооп-игры (Valheim, Don't Starve)
- Небольшие матчи (4-8 игроков)
- Ограниченный бюджет на серверы

Dedicated Server

// Плюсы

- ✓ Защита от читов (сервер проверяет всё)
- ✓ Стабильность (не зависит от игроков)
- ✓ Масштаб: 64-100+ игроков
- ✓ Равные условия для всех

// Минусы

- ✗ Дорого (\$1000-5000/месяц)
- ✗ Сложнее в разработке
- ✗ Нужна инфраструктура (DevOps)

```
#if UNITY_SERVER
    NetworkManager.Singleton
        .StartServer(); // Логика
#else
    NetworkManager.Singleton
        .StartClient(); // Визуал
#endif
```

Примеры игр

CS:GO, Valorant, Overwatch
Fortnite, PUBG, Apex Legends
Большинство AAA мультиплееров

Relay Server гибридная модель

Как работает

1. Один игрок = хост (как Listen Server)
2. Трафик идёт через облачный Relay
3. Relay просто пересылает пакеты

Зачем нужен

- ✓ Решает NAT traversal
- ✓ Игроки не видят IP друг друга
- ✓ Не нужен публичный IP у хоста

Unity Relay Service

Встроен в NGO

50 GB/мес бесплатно

// Недостатки

- ⚠ Хост всё ещё имеет преимущество
- ⚠ Доп. задержка через облако
- ⚠ Читы возможны на стороне хоста

Что такое Authority?

Client Authority

Client: "Я прыгнул на 5м вверх"

Server: "Ок, записываю"

⚠ Клиент может врать!

Server Authority

Client: "Хочу прыгнуть"

Server: "Проверяю... Разрешаю"

✅ Защита от читов, но больше задержка

В **99%** игр используется **Server Authority** для критичных действий
Движение, выстрелы, подбор предметов — всё проверяется сервером

Client-Side Prediction

T=0ms Client нажимает W → **СРАЗУ движается локально**

↓ (отправка на сервер)

T=50ms Server получает → проверяет → одобряет

↓ (ответ клиенту)

T=100ms Client получает подтверждение

Если сервер ОТКЛОНИЛ → Client откатывается («резинка»)

```
if (IsOwner) {  
    transform.position += input * speed; // Локальное предсказание  
    SendMovementServerRpc(pos); // Отправка на сервер  
}
```

Lag Compensation

Проблема

Игрок А стреляет в В (ping 50ms). На экране А — попадание.
Но на сервере В уже сдвинулся → **мимо!**

Решение

1. Сервер хранит историю позиций (500ms)
2. «Откручивает время» на ping стрелка
3. Проверяет коллизию по **СТАРЫМ** позициям

Вот почему в CS:GO иногда убивают «за углом» —
на экране стрелявшего вы ещё были видны

State Sync vs Input Sync

STATE SYNC

Передаём РЕЗУЛЬТАТ

"Я на позиции (5,0,3)"

"Я смотрю на (0,90,0)"

✓ Проще реализовать

✓ Надёжнее

✗ Больше трафика

FPS, TPS, большинство игр

NGO по умолчанию

INPUT SYNC

Передаём НАМЕРЕНИЕ

"Нажал W"

"Нажал Fire"

✓ Меньше трафика

✓ Детерминизм

✗ Сложнее синхронизация

Rocket League, файтинги (GGPO)

Ownership в NGO

```
void Update() {  
    if (!IsOwner) return; // Проверка владения!  
    HandleInput();  
}
```

Концепция

- Каждый NetworkObject принадлежит клиенту
- Только владелец управляет
- Остальные — только наблюдают

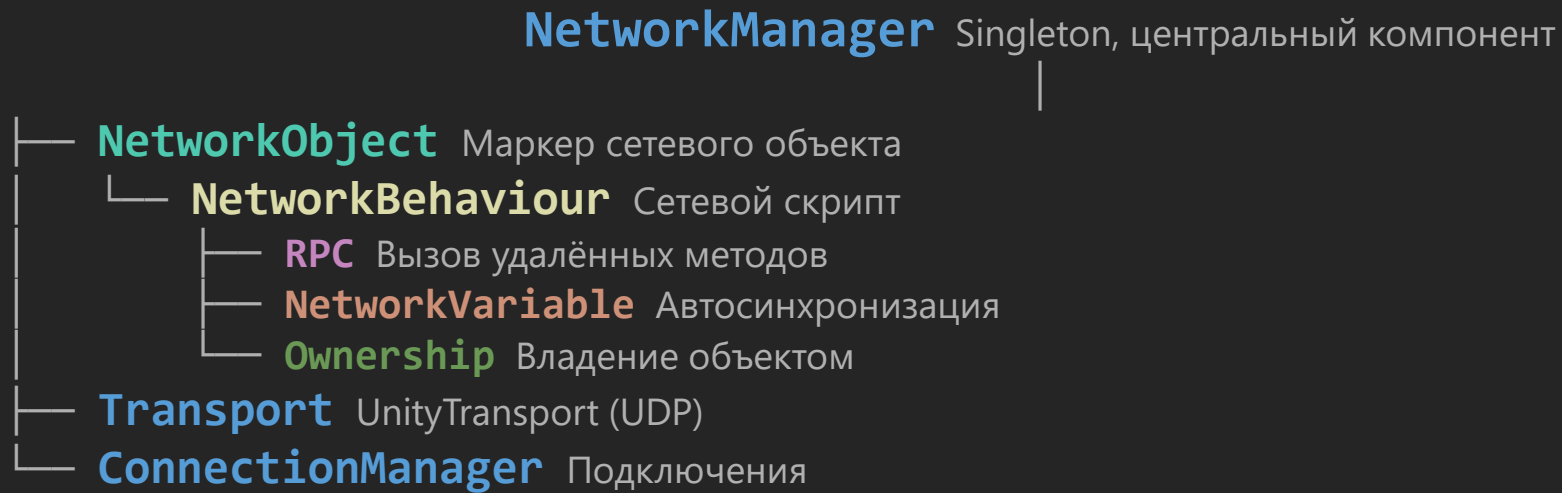
Ключевые флаги

`IsOwner` — я владелец?
`IsServer` — я сервер?
`IsClient` — я клиент?
`IsHost` — сервер + клиент?

Почему NGO?

- 1 **Официальное решение Unity** — Долгосрочная поддержка
- 2 **Бесплатное** — Без ограничений CCU
- 3 **Интеграция** — Relay, Lobby, Matchmaking
- 4 **Современная архитектура** — Учитывает ошибки UNet
- 5 **Open-source** — Можно изучить код

Архитектура NGO



NetworkObject

NetworkObject Component

☐ Synchronize Transform

☐ Destroy With Scene

Network Object Id: 0

Owner Client Id: 0

✓ Когда нужен

- Игрок (Player Prefab)
- Предметы, снаряды

✗ Когда НЕ нужен

- Статика (стены, пол), UI, локальные эффекты

NetworkBehaviour

```
public class PlayerController : NetworkBehaviour {  
    // Автосинхронизация  
    NetworkVariable<int> health = new(100);  
    void Update() {  
        if (!IsOwner) return; // Проверка!  
        HandleInput();  
    }  
    [ServerRpc] // Client → Server  
    void TakeDamageServerRpc(int dmg)  
    [ClientRpc] // Server → All Clients  
    void PlayHitEffectClientRpc()  
}
```

RPC Remote Procedure Calls

[ServerRpc]

Client → Server

Клиент просит сервер выполнить

[ClientRpc]

Server → All Clients

Сервер рассылает всем

```
[ClientRpc]
void PickupItemServerRpc(NetworkObjectReference itemRef) {
    item.Despawn();           // Сервер удаляет
    NotifyPickupClientRpc(); // Всем клиентам
}
```

NetworkVariable

```
NetworkVariable<int> score = new(0);  
// Подписка на изменения  
score.OnValueChanged += (old, new) => UpdateUI();  
// Изменение (только на сервере!)  
if (IsServer) score.Value += points;  
// Автоматически рассылается всем
```

Доступные типы:

```
NetworkVariable<int> <float> <bool> <Vector3>  
NetworkList<T> NetworkDictionary<K,V>
```

Что мы узнали

Топологии

- P2P
- Listen Server
- Dedicated Server
- Relay

Authority

- Server / Client
- Prediction
- Lag Compensation
- State / Input Sync

Unity NGO

- NetworkManager
- NetworkObject
- NetworkBehaviour
- RPC
- NetworkVariable

Что дальше?

Структура практики

1. Настройка NetworkManager

2. Создание UI для подключения

3. Написание ConnectionManager

4. Тестирование двух клиентов